

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-ad96ad1eacfb0a377.jsonl`  
- SHA-256: `caa9350346e6030aaeb0fbcc69e943c84018ec6a309120638c03c28e4adafa9c`  
- Source modified: `2026-07-06T19:18:54+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:17:45.565Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (claims): Tier 2/3 rely on caption text, but ContentItem.text is never populated on the live path ? F2/F4 have an undocumented upstream dependency
severity medium | section Topic preferences ? Tier 2 ('message caption keyword match') and Security stages ? Tier 3 ('caption text scanning', 'caption link extraction' in F4)
summary: Multiple stages depend on caption text (Tier 2 'message caption keyword match', Tier 3 'caption text scanning', F4 'caption link extraction'), and PolicyContext.content_item is the input. But ContentItem.text is documented as 'nullable in Phase 1' and the live handler never sets it: the handler builds a legacy VideoRecord (which has no text field at all) via insert_video, not a ContentItem, so text stays NULL. The ADR does not flag that these stages are inert until a prior track populates content_item.text.
why_it_matters: F2 keyword matching and F4 link extraction would silently match nothing (or F1's PolicyContext wouldn't even carry a caption) until caption ingestion lands. This is a hidden cross-track dependency: the F2/F3/F4 phase table lists dependencies only on F1, but they actually depend on caption text being populated upstream. Without noting it, an implementer wires the stage, sees it 'pass' everything, and ships a no-op.
evidence: models.py:169 ContentItem.text default None, docstring 'nullable in Phase 1' / 'stay NULL until later tracks populate them'. telethon_client.py:258-269 constructs VideoRecord (models.py:383-411, no text field) and calls insert_video ? the ContentItem path (insert_content_item, db.py:1168) is not used by the live handler, and no code reads message caption into text. The ADR's own reference line ('ContentItem as the PolicyContext input') assumes text is available.
rec: Add an explicit dependency note: Tier 2 caption keyword match, Tier 3 caption scanning, and F4 link extraction require ContentItem.text to be populated at ingestion, which today it is not (live path uses VideoRecord/insert_video and never captures the caption). List that ingestion-of-caption work as a prerequisite for F2/F4 in the phasing table, or scope those stages to source-title-only until captions land.

2. assistant (2026-07-06T19:17:48.333Z)

I'll verify this finding by reading the ADR and checking the claims against the codebase.

3. user (2026-07-06T19:17:49.845Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
```

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                      ?
?                               ?                                   ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                  ?
?   duration, resolution, size, codec, mime, GIF/round             ?
?   ? extends today's FilterEngine                                ?
?                               ?                                   ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)               ?
?   source allowlist, keyword match, channel topic tags           ?
?   ? user-expressed interests filter the stream                  ?
?                               ?                                   ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)           ?
?   URL/domain reputation, file metadata sanity,                  ?
?   caption text scanning                                          ?
?   ? cheap checks; no download                                    ?
?                               ?                                   ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                     ?
?   ffprobe, perceptual hash, downloaded content scan             ?
?   ? gated behind explicit user opt-in + budget                  ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
```

```

 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...

```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```

```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
```
```

The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**\*\*No schema migration\*\*** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

## Implementation phasing (future issues)

| Phase   | Scope                   | Dependencies                                                                  |
|---------|-------------------------|-------------------------------------------------------------------------------|
| ***F1** | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler |
| ***F2** | Topic preferences       | `topics` config parsing, source title/caption keyword matching                |
| ***F3** | Source allowlist        | `allowlist` config, skip non-allowlisted sources                              |
| ***F4** | URL/domain reputation   | Domain blacklist, caption link extraction                                     |
| ***F5** | Deep inspection         | ffprobe/perceptual hash, download budget enforcement                          |
| ***F6** | Policy decisions table  | Audit trail storage, explainability UI                                        |
| ***F7** | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace                |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **\*\*design contract\*\*** the phases code against.

## Consequences

### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

## **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## **Alternatives considered**

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

## **Open questions for the decision owner**

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

## **References**

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

### **4. assistant (2026-07-06T19:17:52.869Z)**

Now let me verify the key claims in the finding by checking the codebase.

### **5. user (2026-07-06T19:17:55.041Z)**

main

```
"""A normalized content record discovered from any :class:`Source` (ADR 0011).
```

```
Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
```

```

(``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
"""

model_config = ConfigDict(frozen=True)

id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id")
source_id: int = Field(description="FK ? sources.id")
provider: str = Field(default="telegram", description="Provider namespace")
provider_message_id: str = Field(
 description="Provider-native message id (Telegram message_id, ?)"
)
content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
text: str | None = Field(default=None, description="Caption / post text (nullable in Phase
1)")
link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
media_ref: str | None = Field(default=None, description="Provider media reference")
document_id: str | None = Field(
 default=None,
 description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
)
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None
dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per
account")
matched: bool = False
seen_at: datetime = Field(default_factory=_utcnow)

class Policy(BaseModel):
 """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).

 Generalizes the legacy per-account ``filter_config`` row so one account can
 own multiple policies (enables ADR 0009's per-route filter overrides) and so
 future enrichment/security stages grow into the same ``config_json`` column.
 Exactly one policy per account carries ``is_default=True``; the legacy
 ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
 """

 model_config = ConfigDict(frozen=True)

 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
 account_id: int = Field(description="FK ? accounts.id")
 name: str = Field(description="Policy name; 'default' for the account default policy")

```

## 6. user (2026-07-06T19:17:55.773Z)

```

 and self.min_duration_s > self.max_duration_s
):
 raise ValueError(
 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s
({self.max_duration_s})"
)
 return self

```

```

class VideoRecord(BaseModel):
 """A seen video message + its metadata. Drives dedup and future /search.

```

Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).  
 New code should use :class:`ContentItem`; the ``db`` layer maps  
 ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.  
 """

```
model_config = ConfigDict(frozen=True)

id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
user_id: int
channel_id: int = Field(description="Origin Telegram channel id")
message_id: int = Field(description="Telegram message id within the channel")
document_id: int | None = Field(
 default=None,
 description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
)

Metadata extracted from the Telethon message (may be None if absent).
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None

matched: bool = False
forwarded: bool = False
seen_at: datetime = Field(default_factory=_utcnow)

@property
def dedup_key(self) -> tuple[int, int, int]:
 """The natural key used for deduplication in the ``videos`` table."""
```

## 7. assistant (2026-07-06T19:17:58.595Z)

Confirmed: `ContentItem.text` is nullable and `VideoRecord` has no text field at all. Now let me check the live handler path in telethon\_client.py.

## 8. user (2026-07-06T19:18:00.772Z)

```
if existing is not None:
 log.info(
 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
 "channel=%s msg=%s original_channel=%s original_msg=%s",
 user.id,
 meta.size_bytes,
 meta.duration_s,
 meta.height,
 meta.width,
 channel_id,
 message_id,
 existing.channel_id,
 existing.message_id,
)
 return

Lazily record the channel (best-effort; failures are non-fatal).
try:
 await db.upsert_channel(
 conn,
 Channel(user_id=user.id, channel_id=channel_id, title=_channel_title(event)),
)
except Exception:
 log.warning(
 "failed to upsert channel %s for user %s", channel_id, user.id, exc_info=True
)
```



```

result = engine_holder[ENGINE_KEY].evaluate(meta)
record = VideoRecord(
 user_id=user.id,
 channel_id=channel_id,
 message_id=message_id,
 document_id=document_id,
 duration_s=meta.duration_s,
 width=meta.width,
 height=meta.height,
 size_bytes=meta.size_bytes,
 mime_type=meta.mime_type,
 matched=result.matched,
)
try:
 inserted = await db.insert_video(conn, record)
except sqlite3.IntegrityError:
 # Lost a dedup race: a concurrent delivery of the same file
 # (same document_id or same message instance) won the insert.
 # The three-tier check above already classified this as new, so
 # the collision means another task is processing (or has
 # processed) it ? log and skip.
 log.info(
 "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
 user.id,
 channel_id,
 message_id,
 document_id,
)
 return
log.info(
 "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
 user.id,
 channel_id,
 message_id,

```

#### 9. user (2026-07-06T19:18:02.128Z)

```

9: -> db.insert_video(...) + delivery.forward_to_target(..., target=cfg.delivery_target)
38: from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User, VideoRecord
163: 4. inserts a VideoRecord row,
168: message: Message = event.message # type: ignore[assignment]
258: record = VideoRecord(
271: inserted = await db.insert_video(conn, record)

```

#### 10. assistant (2026-07-06T19:18:05.649Z)

Confirmed: the live handler builds `VideoRecord` (no text field), calls `insert\_video`, never captures a caption. Let me verify insert\_video doesn't populate text from anywhere, and check whether FilterConfig ignores unknown keys (a separate claim in the ADR that's not the focus, but let me confirm the caption text is genuinely never populated).

#### 11. user (2026-07-06T19:18:07.239Z)

```

865: text=row["text"],
1168: async def insert_content_item(conn: aiomysqlite.Connection, item: ContentItem) ->
ContentItem:
1601: async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:

```

#### 12. user (2026-07-06T19:18:12.757Z)

```

async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
 """Deprecated shim: insert into ``content_items`` (ADR 0011).

```

Resolves the legacy ``channel\_id`` to a :class:`Source` row (upserting it

```

if absent), computes the provider-neutral ``dedup_key`` via
:func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
``VideoRecord`` shape is reconstructed from the inserted row.
"""
src = await upsert_source(
 conn,
 Source(
 account_id=video.user_id,
 provider="telegram",
 provider_source_id=str(video.channel_id),
 kind="channel",
 watched=False,
),
)
assert src.id is not None # upsert_source populates the surrogate PK
item = ContentItem(
 account_id=video.user_id,
 source_id=src.id,
 provider="telegram",
 provider_message_id=str(video.message_id),
 content_type="video",
 document_id=str(video.document_id) if video.document_id is not None else None,
 duration_s=video.duration_s,
 width=video.width,
 height=video.height,
 size_bytes=video.size_bytes,
 mime_type=video.mime_type,
 dedup_key=compute_dedup_key(
 provider="telegram",
 document_id=(str(video.document_id) if video.document_id is not None else None),
 source_id=src.id,
 provider_message_id=str(video.message_id),
 size_bytes=video.size_bytes,
 duration_s=video.duration_s,
 height=video.height,
 width=video.width,
),
 matched=video.matched,
 seen_at=video.seen_at,
)
item = await insert_content_item(conn, item)
return _content_item_to_video(item, src, video.user_id, video.channel_id)

```

```

async def mark_forwarded(conn: aiomysqlite.Connection, video_id: int) -> None:
 """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).

```

```

Per-destination delivery state moves to ``delivery_attempts`` (Track C,
#28). For Phase 1 (before Track C lands) there is no equivalent column on
``content_items`` ? the ``matched`` flag alone is sufficient because v1
delivery still targets the single ``delivery_target`` and retried delivery
is harmless (forwards are idempotent). This shim is retained so existing
call sites in ``delivery.py`` keep working without a flag day.
"""

```

```

content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
The row's ``matched`` flag (set at insert time) is the surviving signal.

```

### 13. assistant (2026-07-06T19:18:19.121Z)

Fully confirmed. `insert\_video` constructs a `ContentItem` and never sets `text` ? it defaults to `None`. The entire live path (the only ingestion path per telethon\_client.py:9) leaves `content\_items.text` NULL. There is no code that reads the message caption into text.

Now let me verify the ADR claims that the finding cites ? that Tier 2 and Tier 3 stages depend on caption text, and that the phasing table only lists F1 as the dependency.

From the ADR text I already have:

- Tier 2: "message caption keyword match" (Topic preferences section)
- Tier 3: "caption text scanning" (Tier 4 diagram box) and "check links in captions" (Security stages section)
- F4: "caption link extraction" (phasing table)
- Phasing table: F2 deps = F1, F3 deps = F1, F4 deps = F1 ? none mention caption ingestion.

Let me double-check the ADR does not mention caption ingestion as a prerequisite anywhere.

#### 14. user (2026-07-06T19:18:21.958Z)

```
60:? caption text scanning ?
116:message caption keyword match, and (future) channel topic tags from
127:2. **URL/domain reputation**: check links in captions against a
186:| **F2** ? Topic preferences | `topics` config parsing, source title/caption keyword
matching | F1 |
188:| **F4** ? URL/domain reputation | Domain blacklist, caption link extraction | F1 |
```

#### 15. assistant (2026-07-06T19:18:30.935Z)

The ADR references caption text in 5 places (Tier 2 keyword match, Tier 3 caption scanning, Tier 3 URL-in-captions, F2 caption keyword matching, F4 caption link extraction) but nowhere flags that `ContentItem.text` is unpopulated on the live path, and the phasing table lists only F1 as the dependency for F2/F3/F4.

Let me assess one refutation angle: is this legitimately "out of scope" for a design doc, or does the ADR's own framing make it in-scope? The ADR explicitly claims `ContentItem` "as the PolicyContext input" (References line) and Tier 2 matching is "metadata-only in v1" using "message caption keyword match." Since the ADR treats caption text as an available input for v1/Phase-1 stages (F2, F4 are not deferred like F5/Tier 4), the gap is real and material.

Also worth noting a nuance that slightly strengthens (not weakens) the finding: F2 also has "source title substring match" which *is* available (Source.title exists), so F2 isn't wholly a no-op ? but its caption-keyword half is inert. F4 (URL/domain reputation) is *entirely* caption-dependent ? with no caption text, F4 has nothing to extract links from, making it a complete no-op until caption ingestion lands. That makes F4 the strongest case.

Let me confirm Source has a title field to validate the "source-title-only" fallback in the recommendation.

#### 16. user (2026-07-06T19:18:32.451Z)

```
109: title: str | None = None
123:class Source(BaseModel):
139: title: str | None = None
237: title: str | None = None
```

#### 17. assistant (2026-07-06T19:18:42.344Z)

`Source.title` exists (line 139), so the source-title fallback in the recommendation is viable, and it's also nullable ? but that's a secondary matter. The core finding holds.

Assessment complete. The finding is CONFIRMED:

- **Claim 1** (ContentItem.text nullable, never populated): TRUE. models.py:169 defaults `text=None` with docstring "stay NULL until later tracks populate them"; `insert\_video` (db.py:1601) never sets text; the live handler (telethon\_client.py:258-271) builds a `VideoRecord` with no text field and never reads the caption. No code populates `content\_items.text`.
- **Claim 2** (stages depend on caption text): TRUE. ADR references caption text in Tier 2, Tier 3, F2, and F4.
- **Claim 3** (dependency undocumented; phasing table lists only F1): TRUE. F2/F3/F4 list only F1 as a dependency; no caption-ingestion prerequisite is noted anywhere in the ADR.

Refutation attempts failed. It's not out of scope: the ADR explicitly scopes F2/F4 as Phase-1/v1 (not deferred like Tier 4/F5) and names `ContentItem` as the PolicyContext input, so it is asserting caption text is an available input ? which it is not. The undocumented cross-track dependency is real.

Severity calibration: The finding says medium. This is a genuine hidden cross-track dependency (a gap, not a false factual claim about existing behavior), which maps to medium per the ground rules. It would cause F4 to ship as a complete no-op and F2's caption half to be inert ? a real implementation risk ? but it's a missing dependency note in a draft design doc, not data loss or a false claim. Medium is correct.

**18. user (2026-07-06T19:18:54.363Z)**

Structured output provided successfully